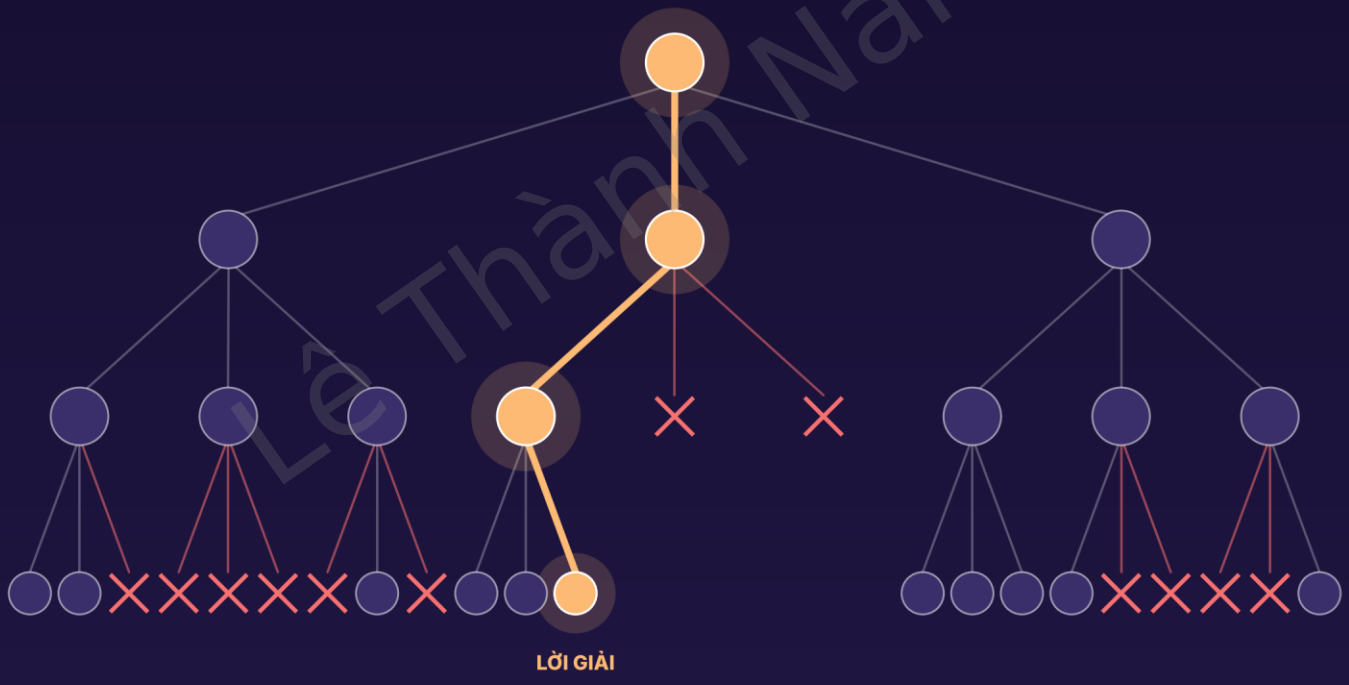
TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

ĐỆ QUY QUAY LUI và NHÁNH CẬN

TRY · RECURSE · UNDO · PRUNE

TẬP 3 · HƯỚNG DẪN GIẢI

Lê Thành Nam



01 Chương 1. ĐỆ QUY
02 Chương 2. QUAY LUI
03 Chương 3. NHÁNH CẬN
04 Chương 4. KĨ THUẬT NÂNG CAO

Từ vét cạn ngây thơ đến cắt nhánh thông minh · 36 bài

Dành cho học sinh THPT ôn luyện học sinh giỏi cấp tỉnh và tiếp cận cấp quốc gia

Tổng hợp và biên soạn: Lê Thành Nam

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải cho 36 bài tập của Tập 2. Mỗi lời giải không chỉ đưa ra chương trình đạt 100 điểm, mà đi lại đúng con đường một học sinh giỏi suy nghĩ trong phòng thi: bắt đầu từ chương trình vét cạn, đếm số trạng thái để thấy bế tắc, rồi phá bế tắc bằng một nhát cắt, một cách tổ chức trạng thái hay một thứ tự duyệt thông minh.

Mọi chương trình trong sách đều đã được kiểm tra hai lần: đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu nhỏ ngẫu nhiên, và chấm trên bộ test chính thức với giới hạn 1 giây, 256 MB. Các con số trong bảng điểm cuối mỗi bài là điểm thực tế mà từng chương trình nhận được, không phải ước lượng. Nhiều bảng điểm có cả những chương trình "gần đúng": vét cạn, quay lui chưa cắt nhánh, cắt nhánh chưa đủ mạnh. Chúng cho em thấy mỗi ý tưởng đáng giá bao nhiêu điểm.

Lê Thành Nam

Lê Thành Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
1. Hiểu bài toán	Diễn đạt lại đề bằng ngôn ngữ của đệ quy và quay lui: trạng thái, lựa chọn, lời giải hoàn chỉnh.
2. Cách ngây thơ	Chương trình vét cạn và số trạng thái nó phải thử.
3. Bế tắc	Vì sao cách hiện tại không vượt qua được subtask lớn; đôi khi kèm một lời giải trung gian.
4. Phá bế tắc	Nhát cắt, cách ghi nhớ hay thứ tự duyệt; hình vẽ chạy tay; lập luận vì sao đúng.
5. Cách tối ưu	Chương trình hoàn chỉnh và phân tích độ phức tạp.
6. Bẫy thường gặp	Những lỗi làm mất điểm hay gặp nhất.
7. Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải mẫu.

Lời khuyên: Đừng đọc lời giải như đọc truyện. Sau mục 3, hãy gấp sách và tự tìm cách phá bế tắc trong ít nhất mười lăm phút. Sau mục 5, hãy tự viết lại chương trình mà không nhìn sách, rồi tự sinh test để đối chiếu với chương trình vét cạn của chính em.

Cách tự đối chiếu với lời giải vét cạn

Ba tệp là đủ: chương trình của em (sol.cpp), chương trình vét cạn (trau.cpp) và một chương trình sinh test ngẫu nhiên nhỏ (gen.cpp, nhận một hạt giống từ dòng lệnh). Đoạn lệnh dưới đây chạy hàng trăm test và dừng lại ở test đầu tiên hai chương trình cho kết quả khác nhau.

‹› Đối chiếu tự động (chạy trên Linux hoặc Git Bash)

```
for i in $(seq 1 500); do
  ./gen $i > test.inp
  ./sol < test.inp > sol.out
  ./trau < test.inp > trau.out
  if ! cmp -s sol.out trau.out; then echo "Khác nhau ở test $i"; break; fi
done
```

Lê Thanh Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CẤU TRÚC MỘT LỜI GIẢI **3**

CHƯƠNG 1. ĐỆ QUY **6**

Bài 1.1 BẬC THANG 6
Bài 1.2 BIỂU THỨC 10
Bài 1.3 VÒNG TRÒN 15
Bài 2.1 KHÔNG LIỀN 20
Bài 2.2 DÃY NGOẶC 23
Bài 2.3 PHÂN HOẠCH 26
Bài 3.1 XẾP CHỮ 30
Bài 3.2 GHÉP SỐ 35
Bài 3.3 TỔ HỢP TỔNG 38

CHƯƠNG 2. QUAY LUI **42**

Bài 4.1 HẬU CẤM 42
Bài 4.2 KHÔNG LẶP 46
Bài 4.3 PHÉP TOÁN 50
Bài 5.1 LÁT GẠCH 54
Bài 5.2 MÃ ĐI TUẦN 58
Bài 5.3 ĐƯỜNG DÀI 63
Bài 6.1 TỔNG SỐ 69
Bài 6.2 MA PHƯƠNG 73
Bài 6.3 MẬT MÃ 79

CHƯƠNG 3. NHÁNH CẬN **85**

Bài 7.1 THANH GỖ 85
Bài 7.2 XẾP ĐỀU 89
Bài 7.3 XẾP HỘP 93
Bài 8.1 GIAO VIỆC 99
Bài 8.2 CHIA TIỆC 104
Bài 8.3 LỊCH LÀM 108
Bài 9.1 CHỌN SÁCH 113
Bài 9.2 BA LÔ GIỚI HẠN 117
Bài 9.3 HAI BA LÔ 123

CHƯƠNG 4. KĨ THUẬT NÂNG CAO **128**

Bài 10.1 SUDOKU NHANH 128
Bài 10.2 TÔ MÀU 134
Bài 10.3 BÈ KẾT 139

Lê Thành Nam

Bài 11.1 XẾP HÀNG 144
Bài 11.2 TRÒ CHƠI 148
Bài 11.3 GIAO HÀNG 152
Bài 12.1 PHÂN SỐ AI CẬP 156
Bài 12.2 XOAY Ô 160
Bài 12.3 LẬT BÁNH 165

TỔNG KẾT: 36 BÀI, MƯỜI HAI Ý TƯỞNG 170

LỜI KẾT 172

Lê Thành Nam

CHƯƠNG 1

ĐỆ QUY

Mỗi bài được phân tích theo cùng một trình tự: hiểu bài toán, cách ngây thơ (kèm chương trình và số trạng thái phải duyệt), chỗ bế tắc, cách phá bế tắc, chương trình tối ưu, các bẫy thường gặp và bảng điểm của từng cách làm. Bảng điểm được đo thật bằng bộ test chấm của sách, với giới hạn 1 giây mỗi test.

BÀI 1.1 BẬC THANG

Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
BACTHANG.INP	BACTHANG.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Bài 1

1. Hiểu bài toán

Đây là bài LEOTHANG của Bài 1 với hai thay đổi: mỗi bước được lên tới k bậc thay vì 2, và một số bậc bị gãy. Gọi f[i] là số cách đứng ở bậc i (đi từ bậc 0). Bước cuối cùng để tới bậc i xuất phát từ một trong các bậc i − 1, i − 2, ..., i − k, nên:

$$f[0] = 1; \quad f[i] = 0 \text{ nếu bậc } i \text{ gãy}; \quad \text{ngược lại } f[i] = f[i-1] + f[i-2] + \ldots + f[i-k]$$

trong đó các số hạng có chỉ số âm được bỏ đi. Đáp án là f[n].

Bậc gãy chỉ làm một việc: **f của nó bằng 0**. Như vậy mọi cách đi qua bậc gãy tự động không được đếm, mà không cần một kiểm tra nào khác.

2. Cách ngây thơ: Chép công thức thành hàm đệ quy

Cách ngây thơ: đệ quy chép công thức, không ghi nhớ

```
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

const long long MOD = 1000000007;
```

```cpp
int n, k;
vector<bool> broken;

// số cách leo từ bậc 0 lên bậc i: đệ quy chép đúng công thức, không ghi nhớ
long long ways(int i) {
    if (i < 0) return 0;
    if (broken[i]) return 0;
    if (i == 0) return 1;
    long long s = 0;
    for (int step = 1; step <= k; step++) s += ways(i - step);
    return s % MOD;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("BACTHANG.INP", "r")) {
        freopen("BACTHANG.INP", "r", stdin);
        freopen("BACTHANG.OUT", "w", stdout);
    }
    int m;
    cin >> n >> k >> m;
    broken.assign(n + 1, false);
    for (int i = 0; i < m; i++) { int b; cin >> b; broken[b] = true; }
    cout << ways(n) << '\n';
    return 0;
}
```

Số lời gọi tăng theo hàm mũ, đúng như bài LEOTHANG: với k = 4 và n = 20 đã là 1 224 513 lời gọi; với k = 2 và n = 40 là 535 828 591 lời gọi. Subtask 1 ($n \le 20, k \le 4$) chạy kịp; từ Subtask 2 trở đi thì không.

3. Bế tắc

Lời giải trung gian: Ghi nhớ, O(n · k)

Mỗi f[i] chỉ phụ thuộc vào các f nhỏ hơn, nên tính lần lượt f[1], f[2], ..., f[n] (hoặc thêm bảng nhớ vào hàm đệ quy). Mỗi f[i] là tổng của k số: O(n · k).

Lời giải trung gian: tính từ dưới lên, O(n · k)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

const long long MOD = 1000000007;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("BACTHANG.INP", "r")) {
        freopen("BACTHANG.INP", "r", stdin);
        freopen("BACTHANG.OUT", "w", stdout);
    }
    int n, k, m;
    cin >> n >> k >> m;
    vector<bool> broken(n + 1, false);
    for (int i = 0; i < m; i++) { int b; cin >> b; broken[b] = true; }
    vector<long long> f(n + 1, 0);
    f[0] = 1;
```

Lê Thành Nam

```
    for (int i = 1; i <= n; i++) {
        if (broken[i]) continue;
        for (int step = 1; step <= k && step <= i; step++) f[i] += f[i - step];   // O(k) mỗi bậc
        f[i] %= MOD;
    }
    cout << f[n] << '\n';
    return 0;
}
```

Subtask 2: $5000 \cdot 5000 = 2{,}5 \cdot 10^7$ phép cộng, kịp. Subtask 3: $10^5 \cdot 10^5 = 10^{10}$ phép cộng: quá chậm.

■ ĐIỂM BẾ TẮC

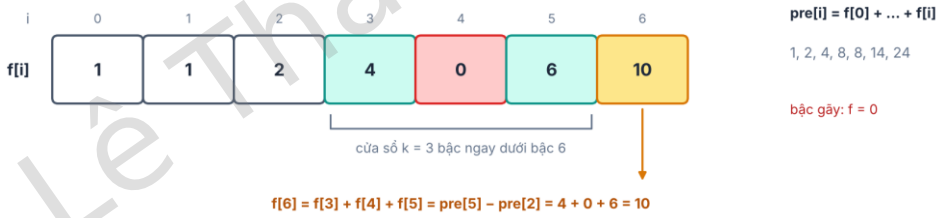
Hai tổng liên tiếp f[i] và f[i + 1] có k − 1 số hạng chung. Tính lại cả k số hạng mỗi lần là lãng phí.

4. Phá bế tắc: Tổng tiền tố

Đặt pre[i] = f[0] + f[1] + ... + f[i]. Khi đó tổng k số liên tiếp là hiệu hai tổng tiền tố:

$$f[i] = pre[i - 1] - pre[i - k - 1] \quad (\text{coi } pre[j] = 0 \text{ khi } j < 0)$$

Mỗi bậc chỉ tốn O(1). Nhớ cập nhật pre[i] = pre[i − 1] + f[i] cho **mọi** bậc, kể cả bậc gãy (khi đó f[i] = 0 và pre[i] = pre[i − 1]).



Hình 1.1.1. Ví dụ 3 (n = 6, k = 3, bậc 4 gãy): f[6] là tổng của cửa sổ ba ô ngay trước nó

Chạy tay ví dụ 3: f = 1, 1, 2, 4, 0, 6, 10 và pre = 1, 2, 4, 8, 8, 14, 24. Chẳng hạn f[5] = pre[4] − pre[1] = 8 − 2 = 6 và f[6] = pre[5] − pre[2] = 14 − 4 = 10.

5. Cách tối ưu

▣ BACTHANG.CPP: Tổng tiền tố, O(n)

```
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

const long long MOD = 1000000007;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("BACTHANG.INP", "r")) {
```

Lê Thành Nam

```
    freopen("BACTHANG.INP", "r", stdin);
    freopen("BACTHANG.OUT", "w", stdout);
  }
  int n, k, m;
  cin >> n >> k >> m;
  vector<bool> broken(n + 1, false);
  for (int i = 0; i < m; i++) { int b; cin >> b; broken[b] = true; }
  // f[i]: số cách đứng ở bậc i; pre[i] = f[0] + ... + f[i]
  vector<long long> f(n + 1, 0), pre(n + 1, 0);
  f[0] = 1; pre[0] = 1;
  for (int i = 1; i <= n; i++) {
    if (!broken[i]) {
      // f[i] = f[i-1] + ... + f[i-k] = pre[i-1] - pre[i-k-1]
      long long s = pre[i - 1] - (i - k - 1 >= 0 ? pre[i - k - 1] : 0);
      f[i] = ((s % MOD) + MOD) % MOD;
    }
    pre[i] = (pre[i - 1] + f[i]) % MOD;
  }
  cout << f[n] << '\n';
  return 0;
}
```

Độ phức tạp: O(n + m) thời gian và bộ nhớ.

6. Bẫy thường gặp

- **Hiệu âm:** pre[i − 1] − pre[i − k − 1] được lấy dư riêng từng số nên có thể âm. Phải cộng thêm MOD trước khi lấy dư.
- **Quên cập nhật pre ở bậc gãy:** pre bị "thủng" một ô, mọi f phía sau đều sai.
- **k lớn hơn i:** chỉ số i − k − 1 âm, phải coi pre là 0 thay vì truy cập ngoài mảng.
- **Bậc n bị gãy:** đáp án 0, công thức tự xử lí nếu không viết thêm trường hợp đặc biệt.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Đệ quy chép công thức, không ghi nhớ	30	0	0	**30**
Ghi nhớ / tính từ dưới lên, O(n·k)	30	30	0	**60**
Tổng tiền tố trượt, O(n)	**30**	**30**	**40**	100

Lê Thành Nam